

Testing Guide

Running Tests

Quick Commands

```
% Run all tests
test_runner()

% Run only unit tests
test_runner('unit')

% Run with verbose output
test_runner('all', 'Verbose', true)

% Run with code coverage
test_runner('all', 'Coverage', true)

% Run specific test file
runtests('tests/unit/test_db_connection.m')
```

Using Quick Scripts

```
run_unit_tests           % Run all unit tests
run_all_tests            % Run all tests
run_tests_with_coverage % Run with coverage report
```

Writing Tests

Basic Test Structure

```
classdef test_my_function < matlab.unittest.TestCase

    methods (TestMethodSetup)
        function setup(testCase)
            % Setup before each test
        end
    end

    methods (TestMethodTeardown)
        function teardown(testCase)
            % Cleanup after each test
        end
    end

    methods (Test)
```

```

        function testBasicFunctionality(testCase)
            % Test basic functionality
            result = my_function(input);
            testCase.verifyEqual(result, expected);
        end
    end
end

```

Using Test Fixtures

```

methods (Test)
    function testWithFixture(testCase)
        % Load test data
        data = TestFixtures.load('sample_survey.csv');

        % Run your function
        result = process_data(data);

        % Load expected output
        expected = TestFixtures.expected_output('expected_result.mat');

        % Verify
        testCase.verifyEqual(result, expected);
    end
end

```

Cleaning Up Test Data

```

methods (TestClassTeardown)
    function cleanupDatabase(testCase)
        % Remove test data from database
        TestFixtures.cleanup('TEST%');
    end
end

```

Test Organization

Unit Tests (**tests/unit/**)

- Test individual functions and classes
- Should be fast (< 1 second per test)
- No external dependencies if possible
- Use mock data
- Tests that need a live database check tests/utils/has_live_db.m and skip themselves when one isn't configured (see e.g. test_db_connection.m)

— there's currently no separate tests/integration/ tier; live-DB-dependent checks live alongside the rest of tests/unit/ and self-skip instead.

Test Fixtures (**tests/fixtures/**)

- sample_data/ - Sample per-survey input CSVs, see tests/fixtures/sample_data/README.md
- TestFixtures.m - Loads/generates mock survey data programmatically
- FakeParser.m, MockBatchConn.m - Lightweight fakes for parser/connection tests

Best Practices

1. One assertion per test (when possible)
2. Test should be independent (no order dependency)
3. Clean up after yourself (remove test data)
4. Use descriptive test names (testCoordinateValidationRejectsInvalidLatitude)
5. Test edge cases (empty inputs, null values, boundaries)
6. Mock external dependencies (don't rely on network/database for unit tests)

Continuous Integration

To run tests automatically:

```
% In your CI script
results = test_runner('all', 'StopOnError', true);
exit(~all([results.Passed]));
```

Usage Examples

```
% === Basic usage ===
startup

% Run all tests
test_runner()

% Run only unit tests with verbose output
test_runner('unit', 'Verbose', true)

% === Generate test fixtures ===
cd tests/fixtures/sample_data
create_sample_fixture
cd ../../..
```

```
% === Run specific tests ===  
runtests('tests/unit/test_db_connection.m')  
runtests('tests/unit/test_fixtures_example.m')  
  
% === Use fixtures in your code ===  
data = TestFixtures.load('sample_survey.csv');  
mock_data = TestFixtures.generate_mock_survey(100);  
  
% === Clean up test data ===  
TestFixtures.cleanup('TEST%');
```